

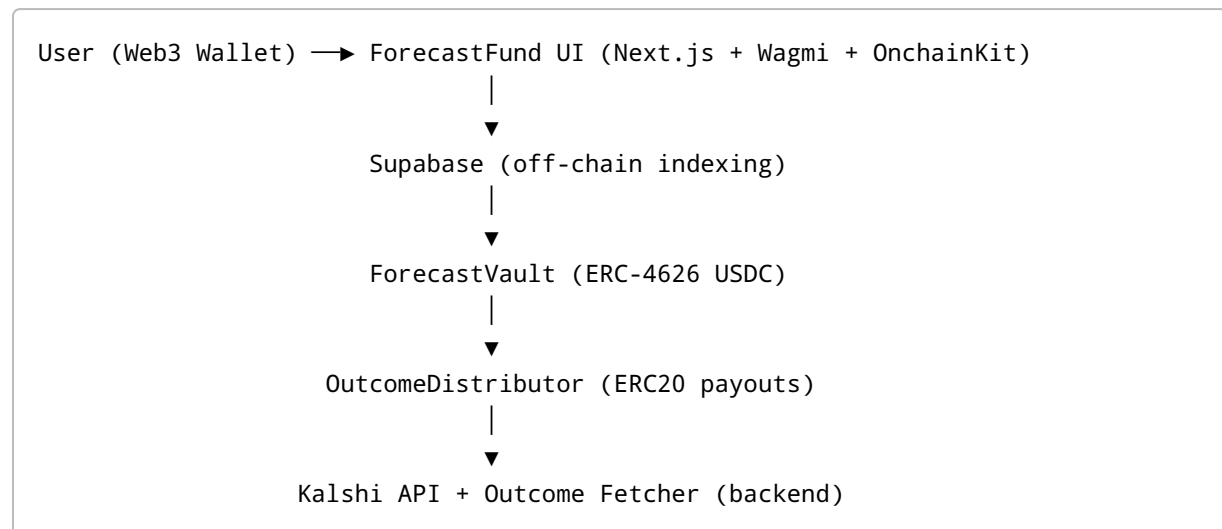
ForecastFund – Flow 2 (USDC Data-Only Integration with Kalshi)

Overview

This document defines **Flow 2**, a compliant and scalable version of ForecastFund that integrates **Kalshi's API as a data and outcome oracle**, without routing user funds or executing trades on Kalshi directly.

Users deposit **Base Sepolia USDC** into ForecastFund smart contracts to participate in prediction-linked yield vaults. The app fetches **Kalshi market data and outcomes** via API and uses those results to calculate on-chain rewards. The system treats Kalshi as a **truth source**, not as a trading venue.

1. System Architecture



Components

- **ForecastVault:** ERC-4626-compatible contract managing user deposits and share issuance.
 - **ForecastRegistry:** Stores EIP-712 signed predictions and nonces.
 - **OutcomeDistributor:** Distributes USDC to winners based on Kalshi outcomes.
 - **Oracle Worker:** Server process that fetches Kalshi results via API, validates them, and calls `distributeBulk()`.
 - **Supabase:** Stores market metadata, prediction records, and leaderboard stats.
-

2. Data Flow

Step 1 – Market Fetching

- The worker queries `https://api.kalshi.com/trade-api/v2/markets` to get open/closed markets.
- Only relevant categories (finance, crypto, macro) are mirrored locally.
- Market data cached in Supabase for the frontend.

Step 2 – User Prediction

- User connects Base wallet and signs an EIP-712 message:

```
type RecordPrediction = {  
  user: address,  
  vaultId: uint256,  
  periodId: uint64,  
  marketId: bytes32, // hash(Kalshi market_id)  
  sideYes: bool,  
  stakePoints: uint256,  
  nonce: uint256,  
  deadline: uint256  
}
```

- Prediction is submitted on-chain via `recordPredictionBySig()`.
- No Kalshi trade occurs — prediction is recorded internally.

Step 3 – Vault Lock & Period Start

- Users deposit USDC via `deposit(assets, receiver)`.
- Vault mints `VaultShares` (ERC-20) to users.
- Period opens; users cannot withdraw until it ends.

Step 4 – Market Resolution via Kalshi

- Oracle Worker polls Kalshi API endpoint `/trade-api/v2/markets/<id>`.
- Once `status = resolved`, the result is written to on-chain Oracle (optional) or Supabase.
- Worker computes per-user accuracy weight using Brier or log scoring.

Step 5 – Settlement & Distribution

- Keeper calls `applySettlement(periodId, pnlDelta)`.
- Worker calls `OutcomeDistributor.distributeBulk(users, amounts, periodId)`.
- Winners receive USDC payouts from vault.

Step 6 – Withdraw or Roll Over

- After settlement, users can `redeem(shares)` for their remaining balance or roll over into the next vault period.
-

3. Technical Design (Key Contracts)

ForecastVault

- **Type:** ERC-4626 vault.
- **Asset:** Base Sepolia USDC (`0x036CbD53842c5426634e7929541eC2318f3dCF7e`).
- **Functions:**
 - `deposit(uint256 assets, address receiver)` → mints shares.
 - `redeem(uint256 shares, address receiver, address owner)` → burns shares.
 - `applySettlement(uint256 periodId, int256 pnlDelta)` → adjusts AUM.
 - `totalManagedAssets()` → view current AUM.

ForecastRegistry

- Stores prediction records (`marketId`, `user`, `stake`, `nonce`).
- Prevents replay via nonces.

OutcomeDistributor

- Takes list of `{user, amount, periodId}` pairs.
 - Executes batched USDC payouts.
 - Callable only by `KEEPER_ROLE`.
-

4. Kalshi Integration Details

API Usage

- **Endpoint:** `https://api.kalshi.com/trade-api/v2/markets`
- **Fields used:** `market_id`, `title`, `status`, `settlement_price`, `yes_bid`, `no_bid`.
- **Polling frequency:** every 15 minutes for open markets; every 2 minutes for closing ones.
- **Auth:** Public key + secret key (if granted via Kalshi Eco).

Compliance Layer

- Data used under Kalshi Eco's "on-chain builder grant" program.
 - Attribution: "Data provided by Kalshi.com."
 - ForecastFund does **not** place trades or issue derivative instruments.
 - Users receive USDC yield based on prediction accuracy only.
-

5. Smart Contract Roles

- **ADMIN_ROLE:** Deploys contracts, manages configurations.
 - **KEEPER_ROLE:** Oracle Worker; triggers settlement and distribution.
 - **DISTRIBUTOR_ROLE:** Smart contract that handles payouts.
 - **PAUSER_ROLE:** Emergency stop authority.
-

6. Vault Tokenomics (Base Sepolia USDC)

- **Deposit asset:** USDC, 6 decimals.
 - **Vault shares:** `ForecastVaultShares (FVS)` minted 1:1 initially.
 - **Yield source:** redistributed pool (not external yield).
 - **Settlement PnL:** computed from Kalshi outcomes and internal accuracy scores.
 - **Fees:** management/performance fees (0–2%) configurable.
-

7. Security

- Contracts audited by third party before mainnet.
 - Off-chain oracle signs results to prevent spoofing.
 - Oracle Worker uses signed attestations for `applySettlement`.
 - Vault is pausable and withdrawable only after each period.
-

8. Deployment Summary

Component	Network	Address	Notes
USDC	Base Sepolia	0x036CbD53842c5426634e7929541eC2318f3dCF7e	6 decimals
ForecastVault	Base Sepolia	TBD	deployed with admin + keeper roles
ForecastRegistry	Base Sepolia	TBD	stores signed predictions
OutcomeDistributor	Base Sepolia	TBD	handles bulk USDC payouts
Oracle Worker	Off-chain	Cloud/VPS	calls Kalshi APIs + Viem scripts

Component	Network	Address	Notes
Supabase	Off-chain	Managed	hosts metadata, leaderboards

9. Legal and Compliance Framing

- Kalshi acts purely as **data and outcome provider**, not as trading counterparty.
- Users participate in **accuracy-based yield programs**, not betting.
- ForecastFund does not issue or settle derivatives, nor hold customer funds beyond vaults.
- ForecastFund uses **Base chain smart contracts** for transparency and immutability.
- Attribution required: "Data provided by Kalshi.com."

10. Roadmap

1. **Phase 1:** Deploy contracts on Base Sepolia; integrate Kalshi API for data-only mode.
2. **Phase 2:** Apply for Kalshi Eco grant and official API key.
3. **Phase 3:** Launch points leaderboard and USDC vault on Base Mainnet.
4. **Phase 4:** Integrate analytics, dynamic weights, and AI-driven scoring.

11. Next Steps for Devs

- [] Deploy contracts to Base Sepolia.
- [] Configure Supabase tables for markets and outcomes.
- [] Implement Oracle Worker to poll and verify results.
- [] Add on-chain call for `applySettlement` + `distributeBulk()`.
- [] Prepare grant submission for Kalshi Eco (with this doc as reference).

Summary:

Flow 2 is a **data-driven, USDC-powered version** of ForecastFund where Kalshi acts as an oracle and ForecastVault handles user funds and payouts transparently on Base. It's legally safer, technically clean, and aligned with the new **Kalshi Eco** initiative.